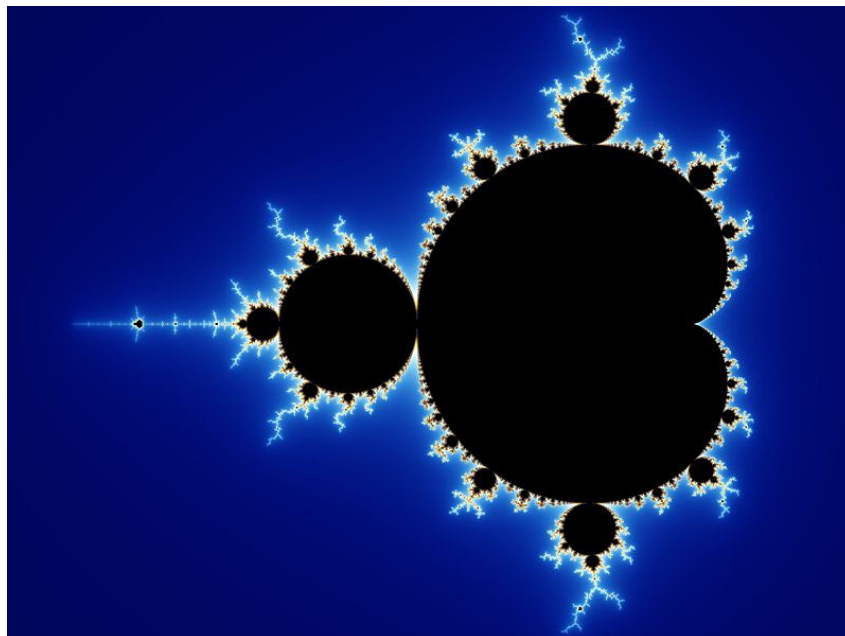


Projet MSE LPSC - 2025-26

Préambule : Dans le cadre du projet LPSC, vous avez reçu une fractale à calculer et à afficher sur la carte SCALP. Le projet est désormais réalisé **obligatoirement par groupe de 2** et s'appuie sur **deux cartes SCALP**. Une première carte affichera la fractale sur son écran et intégrera des calculateurs de fractale. La seconde carte servira d'accélérateur en intégrant des ressources de calcul supplémentaires. Les deux cartes seront connectées entre elles et communiqueront au moyen de **liens série à haute vitesse**. Une **IP Aurora** devra être utilisée pour gérer cette communication. La description qui suit n'est probablement pas le calcul exact que vous devez faire, mais s'en apparente. Il s'agit ci-après de la fractale classique de Mandelbrot.

1 Objectifs

Cet énoncé a pour but d'aider à la conception d'un composant capable d'effectuer les calculs nécessaires à l'affichage de la courbe de Mandelbrot, puis à son intégration dans une architecture répartie sur deux cartes SCALP.



1.1 Description

Nous allons commencer par réaliser un composant capable de calculer le résultat de l'équation de Mandelbrot pour un point donné. Les calculs doivent se faire en virgule fixe.

La courbe de Mandelbrot est construite en appliquant itérativement la fonction $Z_{n+1} = Z_n^2 + C$, où C est le point à évaluer, et $Z_0 = 0$. Le calcul se fait à l'aide de nombres complexes, et si une des itérations génère un point en dehors du cercle centré en $(0, 0)$ de rayon 2, alors le point est en dehors de la courbe. Si tel est le cas, le nombre d'itérations pour atteindre la frontière doit être retourné.

Un nombre maximum d'itérations est défini, et si ce nombre est atteint sans que Z n'ait dépassé le cercle de rayon 2, le nombre d'itérations retourné sera 0. Dans tous les autres cas, le Z résultant de la dernière itération sera retourné, afin de pouvoir afficher correctement la courbe.

1.2 Réalisation du bloc CALCUL

L'entité suggérée du composant pourra être la suivante :

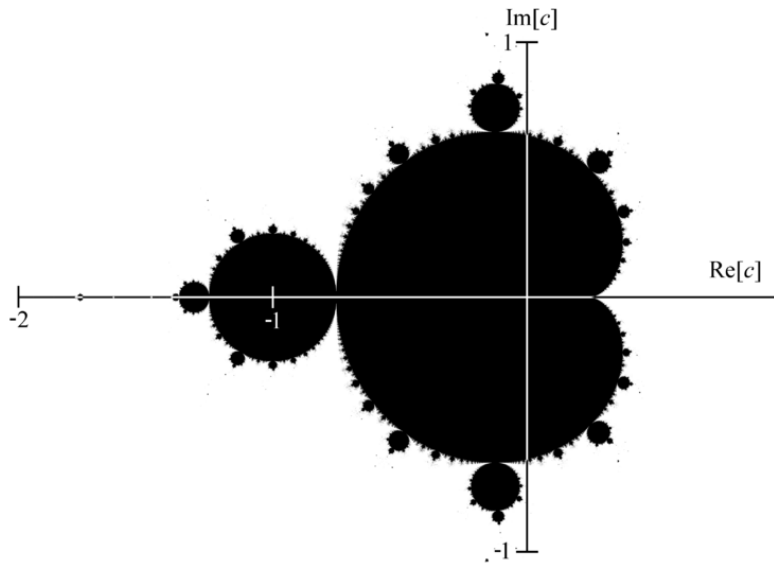
```
entity mandelbrot_calculator is
generic ( comma      : integer := 12; -- nombre de bits après la virgule
         max_iter    : integer := 100;
         SIZE        : integer := 16);
port (
    clk          : in    std_logic;
    rst          : in    std_logic;
    ready        : out   std_logic;
    start        : in    std_logic;
    finished     : out   std_logic;
    c_real       : in    std_logic_vector(SIZE-1 downto 0);
    c_imaginary  : in    std_logic_vector(SIZE-1 downto 0);
    z_real       : out   std_logic_vector(SIZE-1 downto 0);
    z_imaginary  : out   std_logic_vector(SIZE-1 downto 0);
    iterations   : out   std_logic_vector(SIZE-1 downto 0)
);
end mandelbrot_calculator;
```

Les paramètres génériques permettent de définir la taille des opérandes, la place de la virgule, ainsi que le nombre maximum d'opérations.

Le nombre à évaluer est fourni sous la forme d'une partie réelle (*c_real*), et d'une partie imaginaire (*c_imaginary*). Le résultat, donc le dernier Z , l'est également de la même manière, avec (*z_real*, *z_imaginary*). Enfin le nombre d'itérations est également retourné.

Votre calculateur doit indiquer s'il est prêt à effectuer un calcul, en activant *ready*. Si tel est le cas, l'entrée *start* lance un calcul. Lorsque celui-ci est terminé, vous devez activer *finished* pendant un coup d'horloge, en fournissant les sorties *z_** et *iterations*.

Votre implémentation est libre quant à son fonctionnement. Vous pouvez très bien effectuer un calcul complet avant d'en accepter un autre, ou réaliser une structure un peu plus pipelinée. Il vous est vivement suggéré de réaliser un banc de test pour vérifier le bon comportement de votre système.



1.3 Architecture bi-carte SCALP

Le système final devra mettre en oeuvre les deux cartes SCALP avec des rôles distincts :

- **Carte SCALP d’affichage** : elle affiche la fractale sur l’écran, gère l’ordonnancement global du calcul et intègre déjà un premier ensemble de calculateurs de fractale.
- **Carte SCALP d’accélération** : elle intègre des parties de calcul supplémentaires afin d’accélérer le traitement global.
- **Communication inter-cartes** : les deux cartes communiquent par liens série à haute vitesse, avec une **IP Aurora** pour assurer le transport des données.

La répartition exacte des calculs est libre. Vous pouvez par exemple distribuer des lignes, des tuiles d’image ou des ensembles de pixels entre les deux cartes. En revanche, il doit être clairement visible que la première carte calcule et affiche, tandis que la seconde contribue à l’accélération des calculs.

Les données échangées entre les cartes devront être définies avec soin : coordonnées à calculer, indices de pixels, nombre d’itérations ou informations de synchronisation. Les choix de format et de granularité des transferts devront être justifiés.

Une attention particulière devra être portée à la mise en œuvre et à la validation des communications haut débit entre les deux cartes SCALP. Les étudiants devront concevoir des mécanismes permettant d’évaluer les performances de ces échanges.

2 Déroulement du projet

Le projet représente l’équivalent de 7 séances de laboratoire. Il est effectué **en binôme**.

Chaque groupe devra produire une architecture complète mettant en oeuvre les deux cartes SCALP et la communication Aurora. Une progression de travail possible est la suivante :

1. validation en simulation du bloc de calcul élémentaire ;
2. mise en place de la liaison Aurora et validation des échanges inter-cartes ;
3. intégration des calculateurs et de l’affichage sur la carte SCALP principale ;
4. intégration des ressources de calcul supplémentaires sur la seconde carte SCALP ;
5. mesure des performances avec et sans accélération par la seconde carte.
6. Optimisation du calcul, à choix :

- (a) Exécution en parallèle des calculs par la réplication de plusieurs blocs de calculs identiques ;
- (b) Pipeline avec l'accélération des calculs au travers d'un système ayant une horloge plus rapide pour le calculateur ;

Nous vous invitons à implémenter une solution de zoom au travers de différentes valeurs. Des boutons sont à disposition sur la carte. Vous pouvez aussi simplement automatiser l'affichage de quelques ensembles de coordonnées pré-calculées. L'objectif est de visuellement voir la différence de performance avec ou sans l'accélérateur de la seconde carte SCALP, voire des accélérateurs pipeline ou parallèle.

3 Evaluation du projet

Le projet devra être géré au travers d'un environnement de gestion de projet GIT. Le professeur et l'assistant devront disposer des droits de consultation.

Le projet sera évalué de la façon suivante :

- Démonstration du système fonctionnel.
- **Performances des communications haut débit** : Analyse et caractérisation des performances de la communication entre les deux cartes SCALP via les liens série haut débit (IP Aurora). Les étudiants devront mesurer et commenter des métriques telles que le débit effectif, la latence, ainsi que l'impact des communications sur les performances globales du système, intégrant entre autre :
 - les méthodes de mesure utilisées,
 - les résultats expérimentaux (débit, latence),
 - une discussion sur les limitations observées,
 - l'impact sur l'accélération globale du calcul de la fractale.
- Présentation par groupe sous la forme de questions/réponses d'une durée d'environ 10 minutes. Tout support (informatique ou manuscrit) sous la forme de schéma sera le bienvenu entre autres afin de présenter l'architecture globale de votre système, d'indiquer les type d'implémentation choisis (IP core, VHDL, ...) et donner des résultats et analyses de performances.
- Rapport **succinct** écrit à rendre. Ce rapport doit reprendre les différents points cités dans la présentation et bien détailler l'architecture du système. Il doit **surtout** donner des informations sur l'utilisation des ressources du FPGA (en pourcentage d'utilisation des différents blocs) et les analyser. Même chose pour les fréquences d'horloges utilisées et possible. Un effort sera donné sur l'implémentation du calcul de la fractale, entre autres en discutant les résultats d'intégration et d'utilisation dans le FPGA des ressources spécifiques. Un mémoire de moins de maximum 8-10 pages devrait pouvoir être suffisant. Les schémas étant plus que souhaités.
- Participation régulière au projet. L'analyse des *commits* sous GIT pourra être utilisés à ces fins.

Documents informatiques complémentaire à rendre au travers du GIT : code du projet, rapport, documents utilisés pour la présentation.

Accès GIT : Merci d'ajouter le professeur avec le statuts **Reporter** dans votre GIT.

- github.com : fvannel
- gitlab.com : @fabien.vannel
- gitedu.hesge.ch : fabien.vannel

Nommez votre projet avec les noms et prénoms de l'équipe s'il vous plaît. J'ai souvent du mal à retrouver les propriétaires sous des pseudos...

Attention : Le projet sera évalué par groupe, mais la note sera individuelle.